

1. In the notebook, the audio waveform is transformed into time-frequency features before being passed to the keyword spotting model. What is the purpose of converting raw audio into spectrogram/MFCC-style features instead of directly using the raw time-domain waveform?
 - a. Raw audio waveforms are basically just amplitude values over time, which makes them pretty inefficient to work with directly. When we convert to spectrograms and MFCCs, we're extracting the frequency information that actually matters for distinguishing different sounds. MFCCs are especially useful because they're designed to mimic how humans actually hear sounds. They compress the data down to the most important features while keeping what makes "yes" sound different from "no." This is important for the Arduino since it has limited memory and processing power.
2. During full integer quantization, the notebook uses a representative dataset. What is the purpose of the representative dataset, and why is it important for creating an INT8 TensorFlow Lite model?
 - a. The representative dataset helps the quantization process figure out what range of values to expect. When we're converting from 32-bit floats to 8-bit integers, we need to know the typical min and max values so we can scale everything properly. The representative dataset runs through the model and shows what actual activation values look like, so the quantizer can set appropriate ranges. Without this, the model would just guess at the ranges and probably end up with terrible accuracy or values getting clipped.
3. What does `audio_processor.get_data()` do? Refer to `input_data.py`.
 - a. Basically, `audio_processor.get_data()` specifically retrieves a batch of data for a particular set (training, validation, or testing). It takes parameters like how many samples you want, which offset to start at, and which dataset to pull from. The function returns the actual audio features (like the MFCCs) along with their corresponding labels. So when we call it during training, we're basically asking "give me the next batch of preprocessed audio samples and tell me what words they are" so the model can learn from them.
4. In the final model export step, the TensorFlow Lite model is converted into a C/C++ byte array. What are `g_model` and `g_model_len`, and why do they need to be copied correctly into the Arduino source file?
 - a. After we train and quantize the model, we convert it to a C-style byte array that gets written to a file called `kws.cc`. The `g_model` array contains all the model weights, architecture, and everything else the Arduino needs to run inference. `g_model_len` is just the size of that array. We have to copy these from the generated `kws.cc` file into `micro_features_model.cpp` in the Arduino project. If the values don't match or we copy them wrong, the Arduino literally won't know how to interpret the model data.

micro_speech - micro_features_model.cpp | Arduino IDE 2.3.8

Arduino Nano 33 BLE

```
generator.h micro_features_micro_model_settings.cpp micro_features_micro_model_settings.h micro_features_model.cpp ...
1582 0x30, 0x00, 0x00, 0x00, 0x04, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
1583 0x38, 0x00, 0x00, 0x00, 0x14, 0xff, 0xff, 0xff, 0x08, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
1584 0x10, 0x00, 0x00, 0x00, 0x01, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
1585 0x00, 0x00, 0x00, 0x00, 0x01, 0x00, 0x00, 0x00, 0x1d, 0x35, 0x28, 0x38, 0x00, 0x00, 0x00, 0x00,
1586 0x0d, 0x00, 0x00, 0x00, 0x00, 0x66, 0x69, 0x6e, 0x61, 0x6c, 0x5f, 0x66, 0x63, 0x00, 0x00, 0x00, 0x00,
1587 0x5f, 0x62, 0x69, 0x61, 0x73, 0x00, 0x00, 0x00, 0x01, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
1588 0x04, 0x00, 0x00, 0x00, 0x82, 0xff, 0xff, 0xff, 0x00, 0x00, 0x00, 0x01, 0x00, 0x00, 0x00, 0x00,
1589 0x14, 0x00, 0x00, 0x00, 0x14, 0x00, 0x00, 0x00, 0x03, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
1590 0x00, 0x00, 0x00, 0x02, 0x1c, 0x00, 0x00, 0x00, 0xc4, 0xff, 0xff, 0xff, 0x00, 0x00, 0x00, 0x00,
1591 0x0f, 0x00, 0x00, 0x00, 0x52, 0x65, 0x73, 0x68, 0x61, 0x70, 0x65, 0x5f, 0x00, 0x00, 0x00, 0x00,
```

Output Serial Monitor X

Message (Enter to send message to 'Arduino Nano 33 BLE' on '/dev/cu.usbmodem101') New Line 115200 baud

```
Heard unknown (201) @15008ms
Heard unknown (206) @17072ms
Heard unknown (225) @18608ms
Heard yes (203) @19952ms
Heard unknown (217) @21120ms
Heard unknown (208) @23264ms
Heard unknown (204) @24976ms
Heard unknown (201) @27744ms
Heard unknown (214) @33760ms
Heard unknown (220) @35280ms
Heard unknown (216) @36800ms
Heard unknown (206) @41552ms
Heard unknown (208) @45776ms
Heard unknown (207) @51328ms
Heard unknown (202) @62176ms
Heard unknown (213) @77440ms
Heard no (201) @78864ms
Heard unknown (201) @82016ms
Heard unknown (220) @83536ms
Heard unknown (204) @85952ms
Heard unknown (209) @92416ms
```

5.